Process Drift Detection in Event Logs with Graph Convolutional Networks

Leilei Lin[1,4], Yumeng Jin[1], Lijie Wen[3,4], Wenlong Chen[2(✉)], Ying Di[1], Yusong Xu[3], and Jianmin Wang[3,4]

[1] School of Management, Capital Normal University, Beijing, China
leilei_lin@cnu.edu.cn
[2] Information Engineering College, Capital Normal University, Beijing, China
chenwenlong@cnu.edu.cn
[3] School of Software, Tsinghua University, Beijing, China
{wenlj,xys20,jimwang}@tsinghua.edu.cn
[4] Beijing Key Laboratory of Industrial Big Data System and Application, Beijing, China

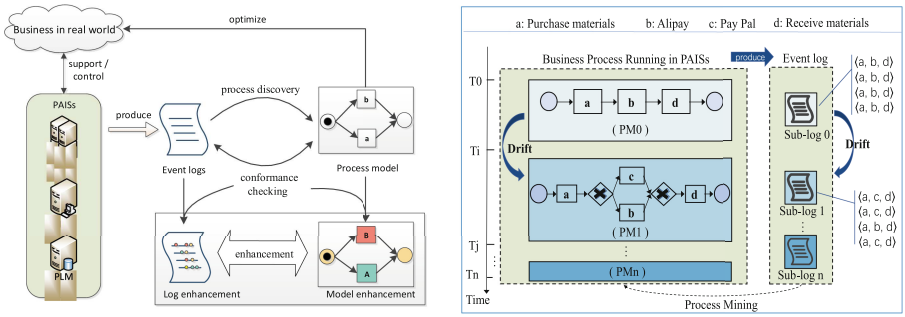
Abstract. The only constant is change. Drift detection in process mining is a family of methods to detect changes by analyzing event logs to ensure the accuracy and reliability of business processes in process-aware information systems (e.g., ERP systems). However, artificial feature selection is still a mountain to climb in existing methods, which requires high domain knowledge for users. In this paper, we propose a novel approach by using **Graph** convolutional networks for **Drifts Detection** in the event log, we name it **GDD**. Specifically, 1) we transform event sequences into two directed graphs by using two consecutive time windows, and construct the line graphs for the directed graphs to capture the orders between different activities; 2) we use graph convolutional networks to capture the features in these graphs, and augment the original graphs with virtual nodes to represent the latent aspects of the graphs; 3) we calculate the distances between virtual nodes, and use the K-means algorithm to find the outliers that are considered as candidate change points. Then, a filter mechanism is used to confirm the actual change points. The experiments on simulated event logs and real-life event logs confirmed the improvements of GDD compared with the baselines.

Keywords: Graph neural networks · Line graph · Process mining · Process changes · Feature propagation

1 Introduction

Process-Aware Information Systems (PAISs) play an important role in business process management (BPM), while process mining serves as a bridge between data mining and business process management [1]. As shown in Fig. 1a, process mining aims to extract valuable information from event logs generated by

X. Wang et al. (Eds.): DASFAA 2023, LNCS 13946, pp. 380–396, 2023.
https://doi.org/10.1007/978-3-031-30678-5_29



(a) Process mining

(b) Drifts detection in process mining

Fig. 1. An example for illustrating the concept drift in process mining

PAISs for configuring and optimizing the ongoing business process. To achieve the aim of process optimization, many classical techniques in process mining have been proposed to support three scenarios: process discovery [2,3] (i.e., extracting process model from an event log without using any apriori information), conformance checking [4] (i.e., monitoring deviations by comparing models and event logs) and process enhancement [5] (i.e., extending or improving an existing model using additional information in event logs). However, these techniques assume processes to be in a steady state. Bose et al. [6] analyzed processes in more than 100 organizations and found that it is very unrealistic to assume that the process being studied is in a steady state. As we know, business processes are continuously evolving in order to adapt to changing circumstances. Some changes are planned, but others occur unexpectedly (e.g. the COVID-19 outbreak) and may remain unnoticed by a majority of stakeholders.

This motivated academics to devise methods and tools that enable business analysts to identify process changes as early as possible. Business process drift detection [7–12] is a family of methods to detect changes by analyzing the event log to ensure the accuracy and reliability of process mining. Figure 1b uses a procurement process as an example to show us the phenomenon of concept drift in process mining. As the business expands, the process PM_0 evolves into the process PM_1 at the time point T_i. Therefore, there is also a change point T_i in the event log generated by the running process. The goal of drift detection is to accurately identify this change point in the log and then divide the log into sub-logs to ensure the accuracy of the process mining.

Still, state-of-the-art methods in this area are limited by artificial feature selection, which will lead to two problems: 1) increase the burden on users. For example, Bose et al. [6,7] propose global features and local features, for users that are asked to identify the features to be used for drift detection, implying that they have some a-priori knowledge of the possible nature of the drift; 2) low accuracy. When users decide to adopt a certain feature to detect changes in the event log, it also means that changes caused by other features will be ignored. To bridge these gaps, we propose GDD (using **G**raph convolutional networks

[14] for **D**rift **D**etection in the event log), which is able to implicitly capture the changing features in the event log by multi-layer convolution as much as possible. The main contributions are summarized as follows.

- We design two types of directed graphs, activity graph and order graph, for capturing change features in the logs.
- A virtual node is added to each directed graph for GCN operation. Then the features of virtual nodes would be treated as the features of directed graphs.
- Inspired by the Feature Fusion that can improve classification results, we similarly aggregate the distances of two kinds of virtual nodes (i.e., activity level and order level) for improving drift detection.
- The K-means algorithm is adopted to find the outliers that are considered as candidate change points. Then, a filtering mechanism is used to locate the real change points, which guarantees the robustness of GDD.

In the experiments, we evaluate the performance of GDD on different datasets. Empirical results show that the proposed method achieved state-of-the-art drift detection in process mining.

2 Related Work

Business processes are prone to change in response to various factors. For this reason, before mining the process model, the moments of change must be found in the event log. Drift detection originated in the field of data mining (a.k.a. concept drift) [13], but is a relatively young research topic that has gained popularity in process mining in the last 10 years.

Bose et al. [6,7] used the strategy of statistical hypothesis testing to determine whether there is a difference between two populations in the logs. The authors define two kinds of features for the user to choose: global features and local features, while global features include relation type count (RC) and relation entropy (RE), and local features include window count and J-Measure. Then, the Hotelling T2 test and K-S test are adopted respectively to detect change points in the event log. Instead of defining a large number of features in [6], a *Run* feature is defined in [8]. The *Run* feature captures accurately concurrency relation in the log. The authors also used hypothesis testing to monitor the change in the distribution of *Run* for detecting drift in the logs. Ostovar et al. [9] substitute the *Alpha* feature for the *Run* feature, because *Alpha* not only captures the concurrent relation in the log, but also the loop relation. Kumar et al. [18] proposed an *Event correlation* feature for drift detection, but the accuracy is low. In [10], the authors used A* algorithm to find the change points. However, this strategy requires expert knowledge to specify the editing cost.

Zheng et al. [11] proposed a new approach to detect drifts based on the DBSCAN algorithm. But, this method is sensitive to noise. The bidirectional testing strategy is used to overcome the noise problem in [12]. Carmona et al. [17] constructed the inequality for detecting drifts based on abstract interpretation.

The authors skillfully transformed the drift detection problem into a numerical computation problem, but one drawback is that drift detection is discontinuous.

Lin et al. [16] detected the change points based on local completeness, namely LCDD. The accuracy of this method is very high, but it can not deal with the noise in the log. Maggi et al. [15] mined the declarative process model based on linear temporal logic, and then the sliding window is used to determine whether the model needs to be updated. When the model is updated, a drift point is found. In [19], the authors brilliantly developed a visualization approach to find out the change points in the logs. Unfortunately, the method requires expert knowledge to determine exactly where the drift occurs.

Table 1. Evaluation capabilities of different approaches

Literature	Strategy	Feature	Evaluation Capability		
			No expert	High accuracy	Robustness
[6,7]	Hypothesis testing	Global and local features	✗	✗	✓
[8]	Hypothesis testing	Run	✓	✗	✓
[9]	Hypothesis testing	Alpha	✓	✗	✓
[10]	A* algorithm	Edit Distance	✗	✗	✓
[11]	DBSCAN	Direct succession	✓	✗	✗
[12]	Bidirectional testing	Directly follows	✓	✗	✓
[15]	Lossy counting	Frequency	✓	✗	✗
[16]	local completetness	Direct succession	✓	✓	✗
[17]	Abstract interpretation	Parikh vector	✓	✗	✗
[18]	Hypothesis testing	Event correlation	✓	✗	✗
[19]	visualization+clustering	Declare constraints	✗	✗	✓
This paper	GCN	Implicit in the graphs	✓	✓	✓

As already remarked, the existing methods have some limitations. Hence, we propose a novel approach for detecting change points based on graph convolutional networks, we name it GDD. As shown in Table 1, we highlight the differences between GDD and the other methods.

3 Preliminaries

In this section, a number of basic definitions will be introduced, and these definitions are borrowed from [8,16].

Definition 1 (Event log). *Let $\mathcal{T}$ be a finite set of activities. An event log $\mathcal{L}$ over $\mathcal{T}$ is defined as $\mathcal{L} = (\mathcal{E}, \mathcal{C}, \gamma, \tau, \succ)$, where $\mathcal{E}$ is a set of events, $\mathcal{C}$ is a set of case identifiers, $\gamma : \mathcal{E} \to \mathcal{C}$ a surjective function relating events to cases, $\tau : \mathcal{E} \to \mathcal{T}$ a function relating events to activities and $\succ \subseteq \mathcal{E} \times \mathcal{E}$ a total order on events.*

Definition 2 (Trace). *Let $\mathcal{T}$ be a finite set of activties and $\mathcal{L} = (\mathcal{E}, \mathcal{C}, \gamma, \tau, \succ)$ a log over $\mathcal{T}$. For all cases $c \in \mathcal{C}$, we define the trace $\sigma_c = \langle e_1, e_2, \cdots, e_n \rangle$ as an event sequence over the event set $\mathcal{E}(\sigma_c) = \{e \in \mathcal{E} | \gamma(e) = c\}$ as all events relating to case c, where n is the number of events in $\mathcal{E}(\sigma_c)$, $e_i \in \mathcal{E}(\sigma_c)$, $i \in \{1, 2, \cdots, n-1\}$ and if $1 \leq i < n$, it holds $e_i \succ e_{i+1}$. The length of the trace σ_c is defined as the number of elements in $\mathcal{E}(\sigma_c)$, which is denoted as $|\sigma_c|$.*

Definitions 1 and 2 state that an event log consists of a set of traces, each capturing the sequence of events for a given case of the process ordered by timestamp. Sometimes a simple event log will be written as a multi-set of traces, which ignores the orders of traces [1]. However, we do not represent event logs as multisets, because the order of each trace would be treated as a time point in drift detection [6,9,16]. It is reasonable to regard the order of trace as a time point, while each trace in logs has a unique ID and start-time. As shown in Fig. 1b, we assume that an event log $\mathcal{L} = [\sigma_0 = \langle a, b, d \rangle, \sigma_1 = \langle a, b, d \rangle, \sigma_2 = \langle a, b, d \rangle, \sigma_3 = \langle a, b, d \rangle, \sigma_4 = \langle a, c, d \rangle, \sigma_5 = \langle a, c, d \rangle, \sigma_6 = \langle a, b, d \rangle, \sigma_7 = \langle a, c, d \rangle]$ has 8 traces. Then, we want to find out the change point T_i (i.e., σ_4) for dividing the event log $\mathcal{L}$ into two sub-logs: one is $\mathcal{L}_0 = [\sigma_0 = \langle a, b, d \rangle, \sigma_1 = \langle a, b, d \rangle, \sigma_2 = \langle a, b, d \rangle, \sigma_3 = \langle a, b, d \rangle]$ and another is $\mathcal{L}_1 = [\sigma_4 = \langle a, c, d \rangle, \sigma_5 = \langle a, c, d \rangle, \sigma_6 = \langle a, b, d \rangle, \sigma_7 = \langle a, c, d \rangle]$.

4 Methodology

To solve the drift detection issue for event logs, we propose GDD, a fully automated method without artificial feature selection. In this section, we will explain the details of our method.

Figure 2 shows an overview of our proposed method. Initially, we convert the traces in the event log into two directed graphs by using two consecutive time windows that would continuously move through the log. Each time the windows move, the directed graphs are updated. Then, we apply the graph convolutional networks for node embedding. After embedding, the high-dimension vector of each node contains the information of its neighbors. A virtual node is added in each directed graph to capture the features of the entire directed graph by connecting to all nodes. Next, we calculate the distances of virtual nodes representing the activity graph and order graph respectively. Finally, the K-means algorithm and a fliter mechanism are used to comfirm the actual change points.

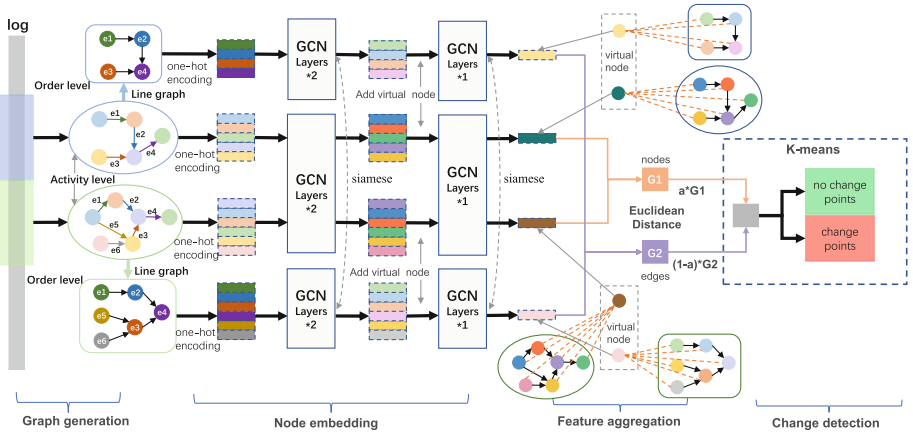


Fig. 2. Overview of our technique named GDD for drift detection

4.1 Graph Generation

From a statistical viewpoint, the problem of business process drift detection can be formulated as follows: *identify a time point when there is a statistically significant difference between the observed process behavior before and after this point* [8]. So, various features are manually selected for hypothesis testing in Fig. 1. However, it is difficult to fully take into account the change features in the event log among these selections. We believe the intuition behind concept drift is that a change happened in a business process must be caused by the changing of activities or the changing of orders among activities. In this setting, a more suitable approach for drift detection is to analyze the differents of the activities and the orders between two sub-logs obtained from consecutive time windows. So, if we can determine that two directed graphs are different where nodes and edges represent activities and orders respectively, a change point can be detected. It is worth emphasizing that this paper, like the references in Table 1, focuses on the control flow perspective while many perspectives can be studied in the business processes (e.g., resource perspective).

Definition 3 (Activity Graph). *Let $\mathcal{T}$ be a finite set of activities in a sub-log $\mathcal{L}$. We define a directed graph $\mathcal{AG} = (\mathcal{V},E)$ be an activity graph, $\mathcal{V}$ represents the set of nodes, E represents the set of ordered pairs of nodes, $E = \mathcal{V} \times \mathcal{V}$, where (1) $\forall n \in \mathcal{T}$, $\exists n \in \mathcal{V}$, and (2) if there is an oder $e_i \succ e_{i+1}$ occurring in a trace σ_c, $\tau(e_i) = a, \tau(e_{i+1}) = b$ and $a, b \in \mathcal{V}$ then $\exists(a, b) \in E$.*

Definition 4 (Order Graph). *Let $\mathcal{AG} = (\mathcal{V},E)$ be an activity graph, we define a line graph $\mathcal{OG} = (\widetilde{\mathcal{V}}, \widetilde{E},)$ be an order graph, where (1) each node $n \in \widetilde{\mathcal{V}}$ represents an edge in E, and (2) each edge $e = (a, b) \in \widetilde{E}$ and $a, b \in \widetilde{\mathcal{V}}$ represents two directed edges $(a, x), (x, b)$ in E and $a, x, b \in \mathcal{V}$.*

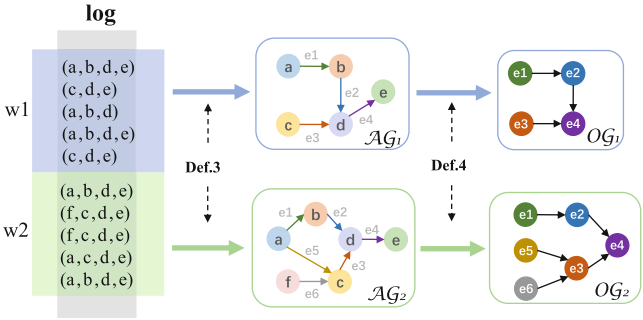


Fig. 3. Activity Graph and Order Graph

To capture the features of activities in the log, we convert the log into a directed graph. Definition 3 states an activity graph $\mathcal{AG}$, where the nodes and edges in $\mathcal{AG}$ represent the activities and orders in the log respectively. Now, the problem of drift detection is transformed into the problem of directed graph convolution. However, if the convolution operation is performed only on the activity graph, it is easy to ignore the information on the edges in $\mathcal{AG}$. Because the graph convolution operation just updates the features of nodes in $\mathcal{AG}$. Inspired by the CensNet [20] that can improve classification performance by convolution with edge-node switching graph. Similarly, we construct a line graph, called $\mathcal{OG}$ in Definition 4, to capture order features in the event log.

An example is given in Fig. 3 to illustrate the conversion process from event logs to $\mathcal{AG}$ and $\mathcal{OG}$. As shown in Fig. 3, we first apply two windows, $w1$ and $w2$, to obtain two sub-logs from the original log. The size of windows is a hyperparameter and $w1 = w2 = 5$ in Fig. 3. Then, the nodes and edges are extracted from the sub-logs to construct two activity graphs based on Definition 3. Specifically, there are five nodes $\mathcal{V} = \{a, b, c, d, e\}$ and four edges $E = \{(a, b), (b, d), (c, d), (d, e)\}$ extracting for the window $w1$ and six nodes $\mathcal{V} = \{a, b, c, d, e, f\}$ and six edges $E = \{(a, b), (b, d), (c, d), (d, e), (a, c), (f, c)\}$ extracting for the window $w2$. Finally, the order graphs are built based on Definition 4, where the nodes in $\mathcal{OG}$ represent the edges in $\mathcal{AG}$.

4.2 Node Embedding

According to Definitions 3 and 4, we have constructed the directed graphs (i.e., $\mathcal{AG}$ and $\mathcal{OG}$) from the event log. After that, some methods have to be used for calculating the differences between these graphs. The traditional graph traversal algorithms, like Depth-First-Search and DeepWalk [21], face the following problems when comparing the difference between two directed graphs: (1) inefficiency. Their efficiency is an obvious drawback when dealing with very complex structures; (2) low accuracy. The traversal algorithms can only compare local features (nodes or second-order proximity) between two directed graphs, they cannot observe the difference between two graphs from the overall topology. At

the same time, graph neural networks [14] have shown extraordinary talents in graph classification in recent years. So, we decide to map the features of the directed graph into high dimensional vectors by applying the multi-layer Graph Convolutional Network [14], where the layer-wise propagation rule is following:

$$H^{(l+1)} = \sigma\left(\widetilde{D}^{-\frac{1}{2}}\widetilde{A}\widetilde{D}^{-\frac{1}{2}}H^{(l)}W^{(l)}\right). \tag{1}$$

Here, $\widetilde{A} = A + I_N$ is the adjacency matrix of the directed graph (i.e, $\mathcal{AG}$ or $\mathcal{OG}$) with added self-connections. I_N is the identity matrix and $\widetilde{D}_{ii} = \sum_j \widetilde{A}_{ij}$. $W^{(l)}$ is a layer-specific trainable weight matrix, $\sigma(\cdot)$ represents an activation function defaulted to the $ReLU(\cdot)$. $H^{(l)} \in \mathbb{Z}^{N\times D}$ is the matrix of activations in the l^{th} layer, while $H^{(0)} = X$.

We know that the matrix I_N is to prevent nodes from losing their own information in the process of feature propagation, while the matrix $\widetilde{D}$ can ensure the normalization of features. But, the trainable weight matrix $W^{(l)}$ is randomly generated, which will seriously impress the similarity computation of the two directed graphs. Hence, we apply the **siamese mechanism** [22] to construct two GCN models with tied weights for $\mathcal{AG}$ convolution as well as $\mathcal{OG}$ convolution. For example, there are two GCN models M and N and two weight matrices $W^M =< w_1^M, w_2^M, \cdots, w_m^M >$ and $W^N =< w_1^N, w_2^N, \cdots, w_n^N >$ which are used to perform convolution in two graphs inputs, we solely focus on siamese architectures with tied weights such that:

$$\begin{cases} W & \equiv N \\ m & \equiv n \\ w_i^M & \equiv w_i^N, \forall i \in [1, 2, \cdots, m] \end{cases}$$

Before graph convolution, each node in $\mathcal{AG}$ and $\mathcal{OG}$ will be assigned a high-dimensional vector based on one-hot encoding, **One_Hot** $(v) \in \mathbb{R}^k$ and k represents the number of the set of nodes in two graphs. So, the initial feature of a directed graph is $X = \{v_1, v_2, \cdots, v_k\} = H^{(0)}$. Then, a two-layer GCN is built for node embedding with an adjacency matrix A. We first calculate $\hat{A} = \widetilde{D}^{-\frac{1}{2}}\widetilde{A}\widetilde{D}^{-\frac{1}{2}}$ in a pre-processing step, and the forward model then takes the simple form:

$$Z = f(X, A) = softmax\left(\hat{A}ReLU\left(\hat{A}XW^{(0)}\right)W^{(1)}\right), \tag{2}$$

where $W^{(0)} \in \mathbb{R}^k$ is an input-to-hidden weight matrix for a hidden layer with k feature maps. The softmax activation function, defined as $softmax(x_i) = \frac{1}{\sum_i exp(x_i)} exp(x_i)$, is applied row-wise.

After convolution by two-layer GCNs, the vector of each node in $\mathcal{AG}$ and $\mathcal{OG}$ encompasses the information of its neighboring nodes. In order to accurately portray the topological information of each directed graph, we add a **virtual node** to each directed graph. The idea is inspired by [23] which has shown the virtual node can facilitate graph representation learning. Therefore, four virtual nodes are added to $\mathcal{AG}_1$, $\mathcal{OG}_1$, $\mathcal{AG}_2$ and $\mathcal{OG}_2$ respectively after two-layer GCNs. Then, all existing nodes in each of the graphs will be connected to the virtual

node to form an expanded graph. The virtual node represents the latent aspects of the expanded graph, which are not immediately available from the attributes and local connectivity structures. Next, we would use a one-layer GCN for the expanded graphs to obtain the vector of each virtual node. So, The representation of the virtual node is then the representation of the entire graph.

4.3 Feature Aggregation

After graph generation and the node embedding, we now obtain the vectors of four virtual nodes $< v_1^{\mathcal{A}}, v_2^{\mathcal{A}}, v_1^{\mathcal{O}}, v_2^{\mathcal{O}} >, v \in \mathbb{R}^{k+1}$ to represent all the features of the four graphs. Among them, $v_1^{\mathcal{A}}$ and $v_2^{\mathcal{A}}$ represent the features of the activity graphs $\mathcal{AG}_1$ and $\mathcal{AG}_2$, $v_1^{\mathcal{O}}$ and $v_2^{\mathcal{O}}$ represent the features of the order graphs $\mathcal{OG}_1$ and $\mathcal{OG}_2$, respectively.

Next, we use the **Euclidean Distance** to calculate the distance between the two graphs separately. An aggregation factor $\alpha \in [0.0, 1.0]$ is introduced to aggregate the distances of the two type graphs to obtain the final distance of the two sub-logs as follows:

$$d(w1, w2) = \alpha \left\| v_1^{\mathcal{A}} - v_2^{\mathcal{A}} \right\| + (1 - \alpha) \left\| v_1^{\mathcal{O}} - v_2^{\mathcal{O}} \right\|, \tag{3}$$

where the value of $d(w1, w2)$ is in [0.0, 1.0], because we apply **Min-Max Normalization** to normalize each distance of two type graphs. The aggregation factor α is set to 0.5 in this paper, which is used to balance the influence of nodes and edges on drift detection.

4.4 Change Detection

In this section, we provide details about change detection based on the distance of two sub-logs. As shown in Fig. 4, there are two steps for detecting change points: (1) we use the K-means algorithm to cluster all points into two classes, where the distant ones are the candidate change points, and (2) we use a filter mechanism to find the actual change points from the candidate points.

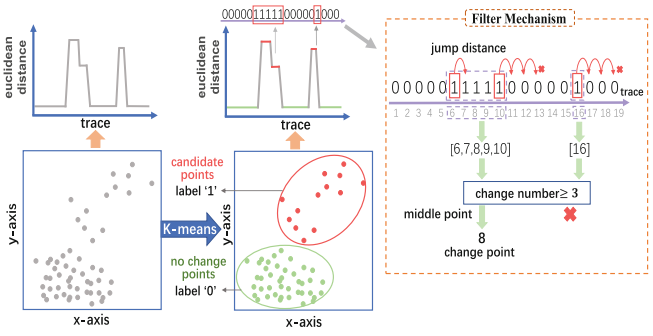


Fig. 4. An example for illustrating the detail of change detection. Two steps are described: K-means algorithm be used to find the candidate points, then filter mechanism be adopted to confirm the actual change points.

K-Means Algorithm. With the sliding of two consecutive windows, many points are obtained from the event log. Each of these points is the intersection of the windows (i.e., the location of the 1st trace in w2). Figure 4 depicts the details of clustering: we first use the K-means algorithm (i.e., K=2) to cluster these points into two classes based on the distance of each point. Second, all points in the class with a large distance will be marked as '1' label, and the points in the other class are marked as '0' label. We believe that changes happened in the event log will result in a larger distance between windows w1 and w2. Therefore, the points with label '1' are considered as candidate change points.

Filter Mechanism. In order to ensure the robustness of our method, we introduce a filter mechanism to improve the performance of GDD. We have found that some situations also cause the distance to become larger rather than just concept drift: (a) Noise traces. This kind of trace is often generated due to employee operation errors or system failures. However, these traces occur less frequently; (b) Execution probability triggered by business branching. There are branching structures such as parallel and alternative in the business process model, and these structures can interfere with the drift judgment. The form triggered by the branch structure is that the labels '0' and '1' appear alternately in the event log. So, we provide a simple-to-implement and efficient mechanism, namely the Filter Mechanism, to reduce the interference of the above situations. The operation steps of the Filter Mechanism are shown below.

The first step is grouping. We use the hyperparameter *jump distance* (abbr., j) to group all the candidate points into different groups. As shown in Fig. 4, assuming $j = 3$, we start from the location of first label '1' and jump three points forward. If there is a point with '1' in the three points, this point will be replaced with the new starting point and continues to jump until no point with label '1' is found. Then, a group is divided successfully. We repeat the above steps until all the points with label '1' are grouped successfully. There are two groups [6,7,8,9,10] and [16] be found in Fig. 4, where each numeral in the groups represents the location of drift.

The second step is filtering. A hyperparameter *change number* (abbr., n) is used to filter out the groups with the number of candidate points less than n. In Fig. 4, the group [16] will be filter out.

The last step is to confirm the actual change points. When the label '1' is no longer appears, it means that the evolution tends to be stable. In other words, a group represents only one evolution. Therefore, we use the middle point to replace all the points in a group. The location of middle point is defined by $middle\ point = group.get(\lfloor \frac{group.size-1}{2} \rfloor)$, where $group.size$ represents the number of candidate points in a group, the function $group.get(i)$ means getting the i-th candidate point from a group and $i = \lfloor \frac{group.size-1}{2} \rfloor$.

Finally, the locations of change points are found (i.e., middle points) through the K-means algorithm and Filter Mechanism. Also, these locations are considered as the point of time when the concept drift occurs in the event log. Using these change points, we are then able to accurately divide the event log into different versions of business processes and provide support for process mining.

5 Experiments

The proposed method (i.e., GDD) is implemented as a stand-alone Python application, and all the code are publicly-available[1].

5.1 Datasets

We conduct experiments on public datasets including PRODRIFT and PTCOST, since they are often used to test the performance of drift detection methods [8,9,12,16].

— PRODRIFT[2], provided by Maaradji et al. [8], is the most cited simulation dataset. This dataset consists of 72 synthetic event logs generated from the loan application process (Fig. 5). The model is composed of different control-flow structures including sequences, loops, alternative and parallel branches. To generate the logs, the base model (Fig. 5) was modified systematically by applying, in turn, one out of 12 simple change patterns described in [8]. These modifications reveal different change patterns, which are categorized into insertion ("I"), resequentialisation ("R") and optionalisation ("O"). Besides, 66 composite change patterns (RIO, ROI, IOR, IRO, OIR, ORI) were created by combining the simple change patterns. Finally, to vary the distance between the drifts, four event logs of sizes 2,500 traces (abbr., 2.5k), 5,000 traces (abbr., 5k), 7,500 traces (abbr., 7.5k) and 10,000 traces (abbr., 10k) were generated for each of the change patterns, and each event log has 9 change points.

— PTCOST[3], provided by Travel Agency, is a real-log, which contains events pertaining to two years (2017–2018) of travel expense claims. The dataset consists of 18246 events, and 2099 traces. As described in the documentation of PTCOST, the processes in 2017 and 2018 are different since 2017 is a pilot year. The information implies that there is a potential drift point for the event log sometime between the end of 2017 and the beginning of 2018.

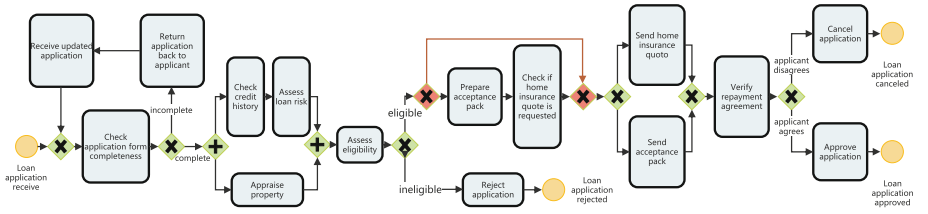


Fig. 5. The process model for the loan application, where the base model does not contain the exclusive gateways(marked as red) but the change pattern CM does. (Color figure online)

[1] https://github.com/lll-lin/GDD.

[2] https://data.4tu.nl/articles/dataset/Business_Process_Drift/12712436.

[3] https://data.4tu.nl/articles/dataset/BPI_Challenge_2020_Prepaid_Travel_Costs/12696722?backTo=/collections/_/5065541.

5.2 Baselines and Metrics

Baselines. We know that many drift detection methods being published in the past years, but it is not possible to assess all the methods. So, we decide to compare GDD with some methods that satisfy the following criteria: (i) the methods can handle noise and their implementations are publicly accessible; (ii) if a method has many extended versions, only the best one will be selected; (iii) the input of the method is the set of traces or the smallest unit is a trace, while the output is the set of process drifts. The application of the three criteria resulted in a selection of baseline methods: **Run** [8], **Alpha** [9] and **BM** [12].

Metrics. Standard F-score metric [6–12] for evaluating the performance among GDD, Run, Alpha and BM is used in our evaluation, where $precision = TP/(TP + FP)$, $recall = TP/(TP + FN)$, and $F\text{-}score = (2 * precision * recall)/(precision + recall)$. TP refers to true positive, FN refers to false negative, and FP refers to false positive. To describe the three variables, a lag period (r) is defined:

- TP: a change point t is detected and there is an actual drift in the integer interval $[t - r, t + r]$.
- FP: a change point t is detected and there is not an actual drift in the integer interval $[t - r, t + r]$.
- FN: there is an actual drift t and there is no change point detected in the integer interval $[t - r, t + r]$.

5.3 Evaluation on Different Parameter Settings

In the first experiment, we evaluate the impact of window sizes w, jump distance j and change number n. We choose the PRODRIFT to test the impact of different parameters in GDD, because this dataset contains different change intervals.

First, the impact of w would be discussed, while j and n are fixed at 3. As shown in Fig. 6a, we test a total of 5 different window sizes ranging from 50 to 500. When the window size is set to $w = 100$, the F-score is the highest, followed by $w = 50$. If the window increases to 200, the performance of GDD starts to drop. When $w = 500$, the F-score is the worst. Therefore, the window size should not be too large, and the accuracy will drop when the window size is larger than the minimal distance between two consecutive drfits. This conclusion is almost identical to that of most current window-based drift detection methods.

Next, Fig. 6b shows us the impact of j with 4 different sizes ($w = 100$ and $n = 3$). We can see that F-scores are highest when $j = 3$. But, it does not perform well when $j = 10$, because it tends to lead to missing the actual drift points for the dataset with small change intervals (e.g., 2.5k).

Last, we set the parameter n with 4 different sizes for observing the F-score of GDD, while w and j are fixed at 100 and 3 respectively. Overall, the parameter of the change number is insensitive in the simulation log. When $n = 3$, the F-score is the most stable, while the F-score decreases slightly when $n = 10$. Because

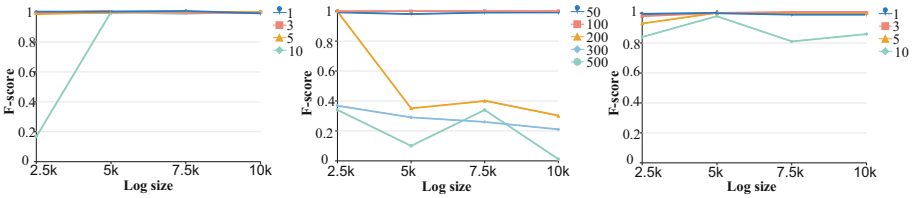


Fig. 6. F-scores under different parameter settings.

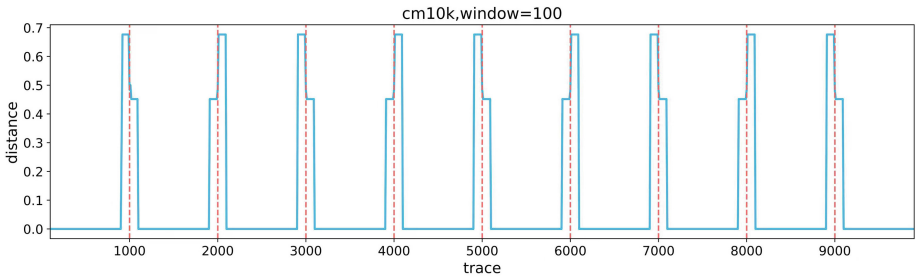
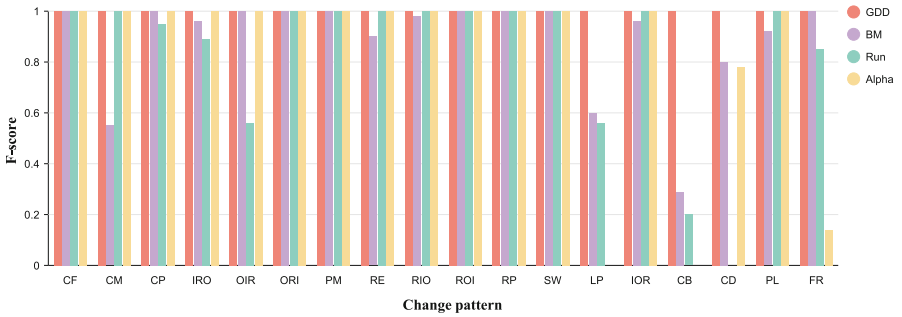


Fig. 7. Visualization of the detection result in the CM10k log and the window size = 100

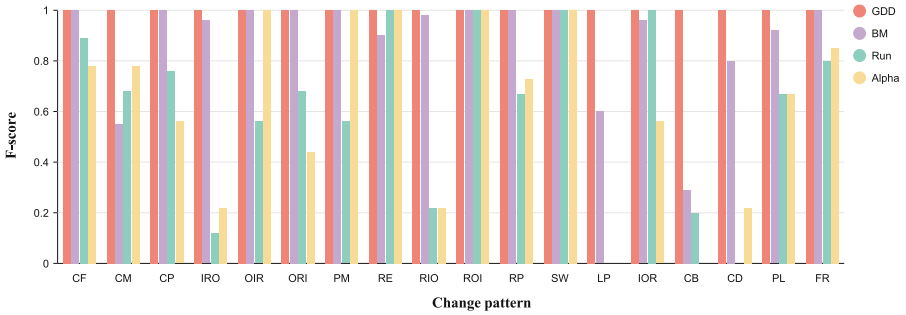
there are no noise data in the simulation log, it also misses a small number of drift points if the value is too large. In the remaining text, we report the results with $w = 100$, $j = 3$ and $n = 3$.

5.4 Comparing with the Baselines

In the second experiment, the accuracy of our proposed method (GDD) and the baselines (Run, Alpha and BM) are compared for each change pattern. Figure 7 depicts the visualization of the detection result by GDD. We can see that the distances between two sub-logs sampled by two consecutive two windows are very high near the actual change point, which matches our initial conjecture. Then, we start by comparing the four methods on the 10k log to see how each method performs specifically on the 18 change pattern, while the delay period decreases from 100 to 50. We find that GDD has perfect F-scores on each pattern. However, the F-scores of most patterns drop dramatically in Run an Alpha. Such as, Fig. 8b reports that when $r = 50$, the F-scores of LP are zero by using Alpha and Run. At the same time, the F-scores of BM on the change patterns of CM, CB and LP also decrease significantly.

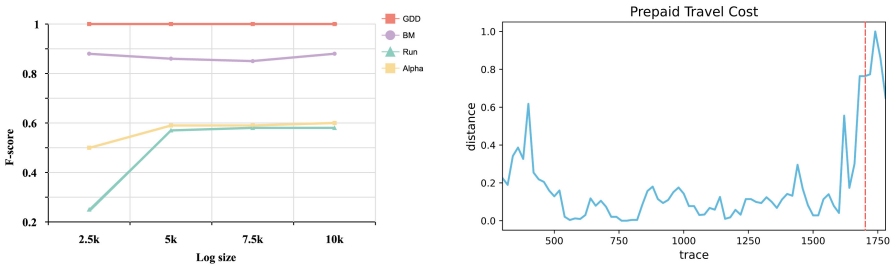


(a) F-score obtained with lag period r=100



(b) F-score obtained with lag period r=50

Fig. 8. Comparison of F-score per change pattern in the 10k log.



(a) F-scores obtained with differnt log sizes (b) The visualization of GDD on real-life log

Fig. 9. Evaluation on simulated logs and real-life log.

Besides, we test the four methods on four different sizes of logs (2.5k, 5k, 7.5k, 10k), and the F-score for each log is the average of 18 change patterns. Figure 9a shows that GDD keeps the highest F-score of 1, while the BM achieves an average F-score of 0.85, Alpha achieves 0.58 and Run achieves 0.51. Hence, the experimental results mean our method is more accurate.

Last, we evaluate our algorithm on the PTCOST dataset (see Fig. 9b). Since the real data have a large number of infrequent behaviors that can easily interfere

with drift detection as noise, we used two sets of parameters:(1) when $w = 100$, $j = 3$ and $n = 3$, we obtain two drift points [325, 1710], of which the drift points 325 is a false positive; (2) when $w = 100$, $j = 3$ and $n = 5$, we obtain only one drift point [1700], and the 1700th trace is the new process that started in January 2018. This result is consistent with the documentation description of the PTCOST. Therefore, the results show that GDD can handle real-life logs very well.

On the aspect of software and hardware, we conduct all tests on an Intel i5-12500H@2.50 GHZ with 16 GB RAM (64 bit), running Windows 10 and python3.7. The average running time on the 10k dataset is 40 s. If you run the test on Goolge Colab, the average time is about 2 min.

5.5 Ablation Study

To clarify the contributions of order graphs, virtual nodes and the filter mechanism, we design $\mathcal{OG}$-$\emptyset$, virtual node-$\emptyset$ and filter mechanism-$\emptyset$. $\mathcal{OG}$-$\emptyset$ and filter mechanism-$\emptyset$ respectively denote the order graphs deletion and the filter mechanism deletion. Virtual node-$\emptyset$ means that we replace the virtual node with the arithmetic mean of each node in the directed graph.

Table 2. Ablation analysis in different logs.

Method	PRODRIFT				PTCOST
	2.5K	5K	7.5K	10K	
$\mathcal{OG}$-$\emptyset$	0.90	0.90	0.92	0.89	0.79
virtual node-$\emptyset$	0.99	0.89	0.91	0.94	0.85
filter mechanism-$\emptyset$	0.99	0.99	0.91	0.90	0.60
GDD	1.0	1.0	1.0	1.0	1.0

As shown in Table 2, the average contributions of order graphs and virtual nodes to the F-score of GDD on the simulation log reach 10%, while the average contribution of the filter mechanism is only 5%. On the contrary, in the real-life log, the contribution of the filtering mechanism is the best, which reaches 40%. Experiments show that the filter mechanism is necessary to improve the robustness of GDD. In addition, the contribution of $\mathcal{OG}$ in the real-log is greater than that in simulation Logs. The essential reason is that the number of edges is far more than the number of nodes in the real business process.

6 Conclusion and Future Work

Drift detection is a valuable topic in process mining, which makes full use of the event log recorded in today's information system to improve business process management. In this paper, we propose a new process drift detection method

(GDD) that can accurately locate the process drift points. Different from previous work, GDD does not rely on artificial feature selection but applies graph convolutional networks to automatically obtain all the change features implied in the logs. We empirically demonstrated that our method outperforms state-of-the-art baselines.

In the future, we will continue to study the following aspects: On the one hand, we aim to propose a way to automatically determine the window size for different logs. On the other hand, we plan to extend the work to support gradual drift detection.

Acknowledgements. The work was supported by the National Key Research and Development Program of China (2018YFB1800403), the general project numbered KM202310028003 of Beijing Municipal Education Commission, the National Natural Science Foundation of China (61872252).

References

1. van der Aalst, W.M.P.: Process Mining: Discovery, Conformance and Enhancement of Business Processes. Springer, New York (2011)
2. Leemans, S.J.J., Fahland, D., van der Aalst, W.M.P.: Discovering block-structured process models from event logs containing infrequent behaviour. In: Lohmann, N., Song, M., Wohed, P. (eds.) BPM 2013. LNBIP, vol. 171, pp. 66–78. Springer, Cham (2014). https://doi.org/10.1007/978-3-319-06257-0_6
3. Guo, Q., Wen, L., Wang, J., Yan, Z., Yu, P.S.: Mining invisible tasks in non-free-choice constructs. In: Motahari-Nezhad, H.R., Recker, J., Weidlich, M. (eds.) BPM 2015. LNCS, vol. 9253, pp. 109–125. Springer, Cham (2015). https://doi.org/10.1007/978-3-319-23063-4_7
4. Sander, J.J., Leemans, A.F.S., Aalst, W.M.P.: Earth moves' stochastic conformance checking. In: International Conference on Business Process Management. Vienna, Austria, pp. 127–143 (2019)
5. Fahland, D., Aalst, W.M.P.: Model repair - aligning process models to reality. Inf. Syst. **47**(1), 220–243 (2015)
6. Bose, R.P.J.C., Aalst, W.M.P., et al.: Dealing with concept drifts in process mining. IEEE Trans. Neural Netw. Learn. Syst. **25**(1), 154–171 (2014)
7. Bose, R.P.J.C., van der Aalst, W.M.P., Žliobaitė, I., Pechenizkiy, M.: Handling concept drift in process mining. In: Mouratidis, H., Rolland, C. (eds.) CAiSE 2011. LNCS, vol. 6741, pp. 391–405. Springer, Heidelberg (2011). https://doi.org/10.1007/978-3-642-21640-4_30
8. Maaradji, A., Dumas, M., La Rosa, M., Ostovar, A.: Fast and accurate business process drift detection. In: Motahari-Nezhad, H.R., Recker, J., Weidlich, M. (eds.) BPM 2015. LNCS, vol. 9253, pp. 406–422. Springer, Cham (2015). https://doi.org/10.1007/978-3-319-23063-4_27
9. Ostovar, A., Maaradji, A., Rosa, M.L., et al.: Detecting drift from event streams of unpredictable business processes. In: International Conference on Conceptual Modeling. Gifu, Japan, pp. 330–346 (2016)
10. Ostovar, A., Leemans, S., Rosa, M.L.: Robust drift characterization from event streams of business processes. ACM Trans. Knowl. Disc. Data **14**(3), 1–57 (2020)

11. Zheng, C., Wen, L., Wang, J.: Detecting process concept drifts from event logs. In: Panetto, H., et al. (eds.) OTM 2017. LNCS, vol. 10573, pp. 524–542. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-69462-7_33
12. Lu, Y., Chen, Q., Poon, S.: A robust and accurate approach to detect process drifts from event streams. In: Polyvyanyy, A., Wynn, M.T., Van Looy, A., Reichert, M. (eds.) BPM 2021. LNCS, vol. 12875, pp. 383–399. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-85469-0_24
13. Widmer, G., Kubat, M.: Learning in the presence of concept drift and hidden contexts. Mach. Learn. **23**(1), 69–101 (1996)
14. Kipf, T.N., Welling, M.: Semi-supervised classification with graph convolutional networks. In: The 5th International Conference on Learning Representations. Toulon, France, pp. 1–14 (2017)
15. Maggi, F.M., Burattin, A., Cimitile, M., Sperduti, A.: Online process discovery to detect concept drifts in LTL-based declarative process models. In: Meersman, R., et al. (eds.) OTM 2013. LNCS, vol. 8185, pp. 94–111. Springer, Heidelberg (2013). https://doi.org/10.1007/978-3-642-41030-7_7
16. Lin, L., Wen, L., Lin, L., Pei, J., Yang, H.: LCDD: detecting business process drifts based on local completeness. IEEE Trans. Serv. Comput. **15**(4), 2086–2099 (2022)
17. Carmona, J., Gavaldà, R.: Online techniques for dealing with concept drift in process mining. In: Hollmén, J., Klawonn, F., Tucker, A. (eds.) IDA 2012. LNCS, vol. 7619, pp. 90–102. Springer, Heidelberg (2012). https://doi.org/10.1007/978-3-642-34156-4_10
18. Kumar, M.V.M., Thomas, L., Annappa, B.: Capturing the sudden concept drift in process mining, pp. 132–143. Algorithms Theories Anal. Event Data, Brussels, Belgium (2015)
19. Yeshchenko, A., Di Ciccio, C., Mendling, J., et al.: Visual drift detection for sequence data analysis of business processes. IEEE Trans. Vis. Comput. Graph. (2021). https://doi.org/10.1109/TVCG.2021.3050071
20. Jiang, X., Ji, P., Li, S.: CensNet: convolution with edge-node switching in graph neural networks. In: International Joint Conferences on Artificial Intelligence (IJCAI), pp. 2656–2662 (2019)
21. Perozzi, B., Al-Rfou, R., Skiena, S.: DeepWalk: online learning of social representations. In: Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 701–710 (2014)
22. Mueller, J., Thyagarajan, A.: Siamese recurrent architectures for learning sentence similarity. In: The Association for the Advancement of Artificial Intelligence (AAAI), pp. 2786–2792 (2016)
23. Pham, T., Tran, T., Dam, H., Venkatesh, S.: Graph classification via deep learning with virtual nodes. arXiv:1708.04357 (2017)